

Loan Default Prediction using Machine Learning

Darshan University — Computer Engineering Department — ML Project

Dataset: Loan Default Prediction (Kaggle) — <https://www.kaggle.com/datasets/nikhil1e9/loan-default>

Objective: Develop a machine learning classification model that predicts whether a borrower will default based on their financial, demographic and loan-related attributes.

Target: Default

Default = 0 → No Default

Default = 1 → Default

In []:

Problem Statement

Banks approve thousands of loans every year. Some customers fail to repay their loans, resulting in financial losses for the institution. The objective of this project is to develop a Machine Learning model that predicts whether a customer is **likely to default on a loan**, using their financial, employment, and demographic details. Early identification of high-risk applicants helps banks make better lending decisions and reduce non-performing assets (NPAs).

Dataset Summary

Item	Description
Dataset Name	Loan Default Prediction
Domain	Banking
Source	Kaggle
Target Variable	Default
Problem Type	Binary Classification (0 = No Default, 1 = Default)

Features used:

- Loan Amount
- Annual Income
- Age

- Employment Type
- Credit Score
- Loan Term
- Interest Rate
- Number of Dependents
- Existing Loans
- Property Area

Initial Observations

- The dataset contains a mix of **numerical** (Loan Amount, Income, Age, Credit Score...) and **categorical** (Employment Type, Property Area...) features.
- Missing values may be present and need to be handled.
- Some numerical features (e.g., Income, Loan Amount) are likely to contain outliers.
- The target variable `Default` is imbalanced in most real-world lending datasets (fewer defaulters than non-defaulters) — this will be checked and considered during evaluation.
- Data preprocessing (cleaning, encoding, scaling) is required before model training.

In []:

```
In [4]: # 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [5]: df = pd.read_csv('Loan_Default.csv')
```

```
In [6]: df
```

Out[6]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	Nur
0	I38PQUQS96	56	85994	50587	520	80	
1	HPSK72WA7R	69	50432	124440	458	15	
2	C1OZ6DPJ8Y	46	84208	129188	451	26	
3	V2KKSFM3UN	32	31713	44799	743	0	
4	EY08JDHTZP	60	20437	9139	633	8	
...	...	...	...	...	...	...	...
255342	8C6S86ESGC	19	37979	210682	541	109	
255343	98R4KDHNNND	32	51953	189899	511	14	
255344	XQK1UUUNGP	56	84820	208294	597	70	
255345	JAO28CPL4H	42	85109	60575	809	40	
255346	ZTH91CGL0B	62	22418	18481	636	113	

255347 rows × 18 columns



In []:

In [4]:

```
df.head() # First 5 rows
```

Out[4]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCredit
0	I38PQUQS96	56	85994	50587	520	80	
1	HPSK72WA7R	69	50432	124440	458	15	
2	C1OZ6DPJ8Y	46	84208	129188	451	26	
3	V2KKSFM3UN	32	31713	44799	743	0	
4	EY08JDHTZP	60	20437	9139	633	8	



In [5]:

```
df.tail() # Last 5 rows
```

Out[5]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	Nur
255342	8C6S86ESGC	19	37979	210682	541		109
255343	98R4KDHNNND	32	51953	189899	511		14
255344	XQK1UUUNGP	56	84820	208294	597		70
255345	JAO28CPL4H	42	85109	60575	809		40
255346	ZTH91CGL0B	62	22418	18481	636		113

In [6]:

```
df.sample() # Random Rows
```

Out[6]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	Nur
165964	XGVWSFBKTD	21	43713	133159	581		60

In [7]:

```
df.sample(5) # Random 5 Rows
```

Out[7]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	Nur
17225	ZZR97F18T2	67	91837	125398	680		83
203144	C9XILN34L2	47	98616	52945	313		30
49627	N91K3B1QRO	25	63654	104027	399		20
91616	HWLD0EASZ5	31	143193	188498	583		83
125095	VUZ11WOVL4	20	59062	246301	348		35

In [8]:

```
df.shape # Dataset Shape
```

Out[8]: (255347, 18)

In [9]:

```
print("Rows :", df.shape[0])
print("Columns :", df.shape[1])
```

```
Rows : 255347
Columns : 18
```

In [8]:

```
df.index # Dataset Index
```

Out[8]: RangeIndex(start=0, stop=255347, step=1)

In [12]:

```
df.info() # Dataset Information
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 255347 entries, 0 to 255346
Data columns (total 18 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   LoanID                255347 non-null object
 1   Age                   255347 non-null int64
 2   Income                255347 non-null int64
 3   LoanAmount            255347 non-null int64
 4   CreditScore           255347 non-null int64
 5   MonthsEmployed        255347 non-null int64
 6   NumCreditLines         255347 non-null int64
 7   InterestRate          255347 non-null float64
 8   LoanTerm              255347 non-null int64
 9   DTIRatio              255347 non-null float64
10   Education              255347 non-null object
11   EmploymentType        255347 non-null object
12   MaritalStatus         255347 non-null object
13   HasMortgage           255347 non-null object
14   HasDependents         255347 non-null object
15   LoanPurpose           255347 non-null object
16   HasCoSigner           255347 non-null object
17   Default               255347 non-null int64
dtypes: float64(2), int64(8), object(8)
memory usage: 35.1+ MB

```

```
In [11]: df.dtypes # Data Types
```

```

Out[11]: LoanID                object
Age                   int64
Income                int64
LoanAmount            int64
CreditScore           int64
MonthsEmployed        int64
NumCreditLines         int64
InterestRate          float64
LoanTerm              int64
DTIRatio              float64
Education              object
EmploymentType        object
MaritalStatus         object
HasMortgage           object
HasDependents         object
LoanPurpose           object
HasCoSigner           object
Default               int64
dtype: object

```

```
In [13]: df.columns # Column Names
```

```

Out[13]: Index(['LoanID', 'Age', 'Income', 'LoanAmount', 'CreditScore',
                'MonthsEmployed', 'NumCreditLines', 'InterestRate', 'LoanTerm',
                'DTIRatio', 'Education', 'EmploymentType', 'MaritalStatus',
                'HasMortgage', 'HasDependents', 'LoanPurpose', 'HasCoSigner',
                'Default'],
                dtype='object')

```

```
In [14]: df.nunique() # Number of Unique Values
```

```
Out[14]: LoanID      255347
Age           52
Income       114620
LoanAmount   158729
CreditScore  550
MonthsEmployed 120
NumCreditLines 4
InterestRate 2301
LoanTerm      5
DTIRatio      81
Education     4
EmploymentType 4
MaritalStatus 3
HasMortgage   2
HasDependents 2
LoanPurpose   5
HasCoSigner   2
Default       2
dtype: int64
```

```
In [15]: df.describe() # Categorical Statistical Summary
```

```
Out[15]:
```

	Age	Income	LoanAmount	CreditScore	MonthsEmployed
count	255347.000000	255347.000000	255347.000000	255347.000000	255347.000000
mean	43.498306	82499.304597	127578.865512	574.264346	59.541976
std	14.990258	38963.013729	70840.706142	158.903867	34.643376
min	18.000000	15000.000000	5000.000000	300.000000	0.000000
25%	31.000000	48825.500000	66156.000000	437.000000	30.000000
50%	43.000000	82466.000000	127556.000000	574.000000	60.000000
75%	56.000000	116219.000000	188985.000000	712.000000	90.000000
max	69.000000	149999.000000	249999.000000	849.000000	119.000000

```
In [20]: df.describe(include='object') # Categorical Statistical Summary
```

```
Out[20]:
```

	LoanID	Education	EmploymentType	MaritalStatus	HasMortgage	HasDe
count	255347	255347	255347	255347	255347	
unique	255347	4	4	3	2	
top	ZTH91CGL0B	Bachelor's	Part-time	Married	Yes	
freq	1	64366	64161	85302	127677	

```
In [13]: df['Default'].value_counts() # Target Distribution
```

```
Out[13]: Default
0      225694
1       29653
Name: count, dtype: int64
```

```
In [14]: # Target Percentage
df['Default'].value_counts(normalize=True) * 100
```

```
Out[14]: Default
0      88.387175
1      11.612825
Name: proportion, dtype: float64
```

```
In [ ]: # 88.39% → No Default
# 11.61% → Default
```

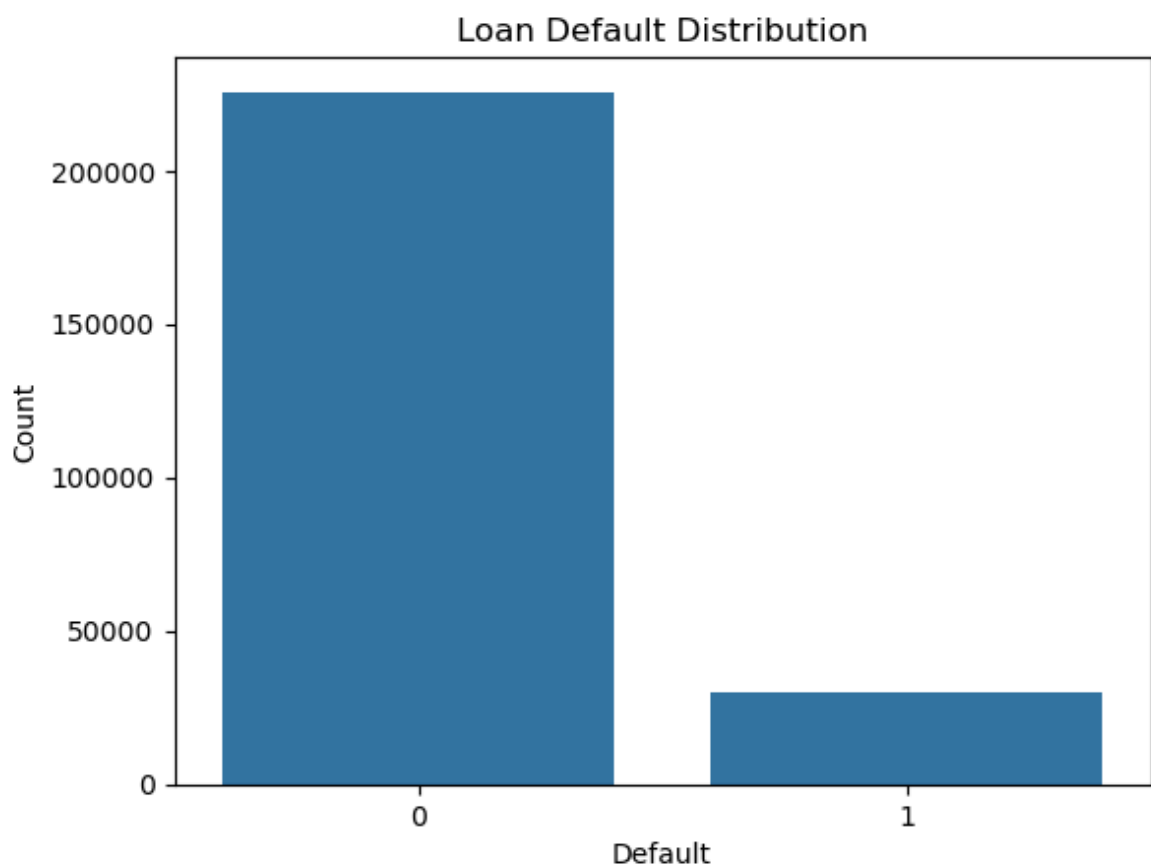
```
In [ ]:
```

```
In [15]: # Target Visualization
sns.countplot(x='Default', data=df)

plt.title('Loan Default Distribution')
plt.xlabel('Default')
plt.ylabel('Count')

plt.show()

# No Default vs Default
```



```
In [16]: # Missing Values
df.isnull().sum()
```

```
Out[16]: LoanID      0
         Age         0
         Income      0
         LoanAmount  0
         CreditScore  0
         MonthsEmployed 0
         NumCreditLines 0
         InterestRate 0
         LoanTerm     0
         DTIRatio     0
         Education    0
         EmploymentType 0
         MaritalStatus 0
         HasMortgage  0
         HasDependents 0
         LoanPurpose  0
         HasCoSigner  0
         Default      0
         dtype: int64
```

```
In [44]: # Missing Value Percentage
         df.isnull().mean() * 100
```

```
Out[44]: LoanID      0.0
         Age         0.0
         Income      0.0
         LoanAmount  0.0
         CreditScore  0.0
         MonthsEmployed 0.0
         NumCreditLines 0.0
         InterestRate 0.0
         LoanTerm     0.0
         DTIRatio     0.0
         Education    0.0
         EmploymentType 0.0
         MaritalStatus 0.0
         HasMortgage  0.0
         HasDependents 0.0
         LoanPurpose  0.0
         HasCoSigner  0.0
         Default      0.0
         dtype: float64
```

```
In [45]: # Duplicate Rows
         df.duplicated().sum()
```

```
Out[45]: np.int64(0)
```

```
In [52]: # Numerical Median
         df.median(numeric_only=True)
```

```
Out[52]: Age                43.00  
Income            82466.00  
LoanAmount       127556.00  
CreditScore      574.00  
MonthsEmployed   60.00  
NumCreditLines   2.00  
InterestRate     13.46  
LoanTerm         36.00  
DTIRatio         0.50  
Default          0.00  
dtype: float64
```

```
In [53]: # Numerical Mode  
df.mode(numeric_only=True).iloc[0]
```

```
Out[53]: Age                55.00  
Income            69492.00  
LoanAmount       95419.00  
CreditScore      630.00  
MonthsEmployed   56.00  
NumCreditLines   2.00  
InterestRate     14.92  
LoanTerm         48.00  
DTIRatio         0.67  
Default          0.00  
Name: 0, dtype: float64
```

```
In [54]: # Minimum Values  
df.min(numeric_only=True)
```

```
Out[54]: Age                18.0  
Income            15000.0  
LoanAmount       5000.0  
CreditScore      300.0  
MonthsEmployed    0.0  
NumCreditLines    1.0  
InterestRate      2.0  
LoanTerm         12.0  
DTIRatio          0.1  
Default          0.0  
dtype: float64
```

```
In [55]: # Maximum Values  
df.max(numeric_only=True)
```

```
Out[55]: Age                69.0  
Income           149999.0  
LoanAmount      249999.0  
CreditScore     849.0  
MonthsEmployed  119.0  
NumCreditLines   4.0  
InterestRate    25.0  
LoanTerm        60.0  
DTIRatio        0.9  
Default         1.0  
dtype: float64
```

```
In [56]: # Standard Deviation  
df.std(numeric_only=True)
```

```
Out[56]: Age                14.990258
Income            38963.013729
LoanAmount        70840.706142
CreditScore       158.903867
MonthsEmployed    34.643376
NumCreditLines    1.117018
InterestRate      6.636443
LoanTerm          16.969330
DTIRatio          0.230917
Default           0.320379
dtype: float64
```

```
In [10]: # Memory Usage
df.memory_usage(deep=True)
```

```
Out[10]: Index                132
LoanID              15065473
Age                 2042776
Income              2042776
LoanAmount          2042776
CreditScore         2042776
MonthsEmployed      2042776
NumCreditLines      2042776
InterestRate         2042776
LoanTerm            2042776
DTIRatio            2042776
Education           14557535
EmploymentType      15128774
MaritalStatus       14299453
HasMortgage         13150374
HasDependents       13150439
LoanPurpose         14044522
HasCoSigner         13150398
Default             2042776
dtype: int64
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

Week 2 — Data Cleaning, Preprocessing & EDA

```
In [7]: # Create Clean Copy
df_clean = df.copy()
```

```
In [8]: # Check Missing Values
print("Total Missing Values:",
      df_clean.isnull().sum().sum())

df_clean.isnull().sum()
```

Total Missing Values: 0

```
Out[8]: LoanID      0
        Age        0
        Income     0
        LoanAmount  0
        CreditScore 0
        MonthsEmployed 0
        NumCreditLines 0
        InterestRate 0
        LoanTerm    0
        DTIRatio    0
        Education   0
        EmploymentType 0
        MaritalStatus 0
        HasMortgage 0
        HasDependents 0
        LoanPurpose 0
        HasCoSigner 0
        Default     0
        dtype: int64
```

```
In [9]: # Check Duplicate Rows
        print("Duplicate Rows:",
              df_clean.duplicated().sum())
```

Duplicate Rows: 0

```
In [10]: # Remove Duplicate Rows
         df_clean.drop_duplicates(inplace=True)
```

```
In [7]: # Check Data Types
         df_clean.dtypes
```

```
Out[7]: LoanID      object
        Age        int64
        Income     int64
        LoanAmount  int64
        CreditScore int64
        MonthsEmployed int64
        NumCreditLines int64
        InterestRate float64
        LoanTerm    int64
        DTIRatio    float64
        Education   object
        EmploymentType object
        MaritalStatus object
        HasMortgage  object
        HasDependents object
        LoanPurpose  object
        HasCoSigner  object
        Default     int64
        dtype: object
```

```
In [11]: # Identify Numerical Columns
         numeric_cols = df_clean.select_dtypes(include=np.number).columns.tolist()

         numeric_cols.remove('Default')

         print(numeric_cols)
```

```
['Age', 'Income', 'LoanAmount', 'CreditScore', 'MonthsEmployed', 'NumCreditLines', 'InterestRate', 'LoanTerm', 'DTIRatio']
```

```
In [12]: # Remove LoanID
df_clean.drop('LoanID', axis=1, inplace=True)
```

```
In [13]: # Check Shape After Removing LoanID
print("Rows      :", df_clean.shape[0])
print("Columns   :", df_clean.shape[1])
```

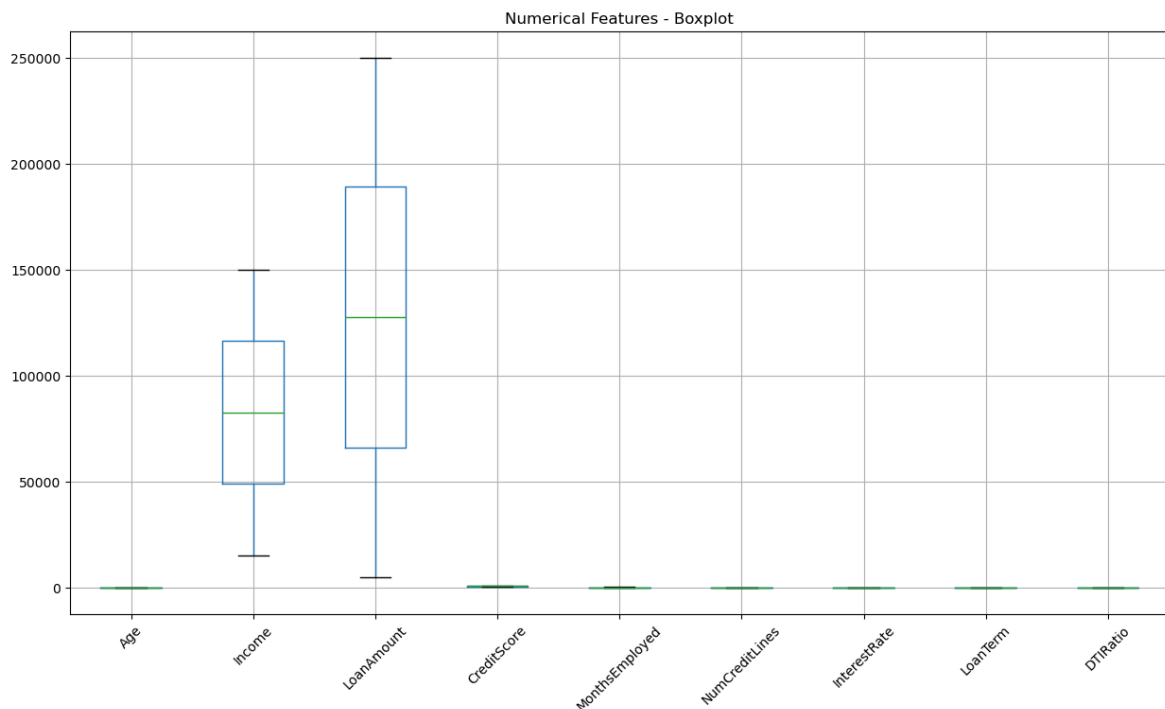
```
Rows      : 255347
Columns   : 17
```

```
In [14]: # Numerical Boxplots
plt.figure(figsize=(15, 8))

df_clean[numeric_cols].boxplot()

plt.xticks(rotation=45)
plt.title('Numerical Features - Boxplot')

plt.show()
```



IQR Method

```
In [29]: # IQR Outlier Detection
for col in numeric_cols:

    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
```

```
outliers = df_clean[
    (df_clean[col] < lower) |
    (df_clean[col] > upper)
]

print(col, ":", len(outliers))
```

```
Age : 0
Income : 0
LoanAmount : 0
CreditScore : 0
MonthsEmployed : 0
NumCreditLines : 0
InterestRate : 0
LoanTerm : 0
DTIRatio : 0
```

Outliers were checked using the IQR method. No IQR-based outliers were detected in the numerical features. Therefore, no rows were removed due to outliers.

In []:

Categorical Variables

```
In [15]: categorical_cols = df_clean.select_dtypes(include='object').columns

print(categorical_cols)
```

```
Index(['Education', 'EmploymentType', 'MaritalStatus', 'HasMortgage',
      'HasDependents', 'LoanPurpose', 'HasCoSigner'],
      dtype='object')
```

```
In [16]: # Categorical Value Counts
for col in categorical_cols:
    print("-----\n", col)
    print(df_clean[col].value_counts())
```

```
-----  
Education  
Education  
Bachelor's      64366  
High School    63903  
Master's       63541  
PhD            63537  
Name: count, dtype: int64  
-----
```

```
-----  
EmploymentType  
EmploymentType  
Part-time      64161  
Unemployed     63824  
Self-employed  63706  
Full-time      63656  
Name: count, dtype: int64  
-----
```

```
-----  
MaritalStatus  
MaritalStatus  
Married        85302  
Divorced       85033  
Single         85012  
Name: count, dtype: int64  
-----
```

```
-----  
HasMortgage  
HasMortgage  
Yes            127677  
No             127670  
Name: count, dtype: int64  
-----
```

```
-----  
HasDependents  
HasDependents  
Yes            127742  
No             127605  
Name: count, dtype: int64  
-----
```

```
-----  
LoanPurpose  
LoanPurpose  
Business       51298  
Home           51286  
Education      51005  
Other          50914  
Auto           50844  
Name: count, dtype: int64  
-----
```

```
-----  
HasCoSigner  
HasCoSigner  
Yes            127701  
No             127646  
Name: count, dtype: int64  
-----
```

In []:

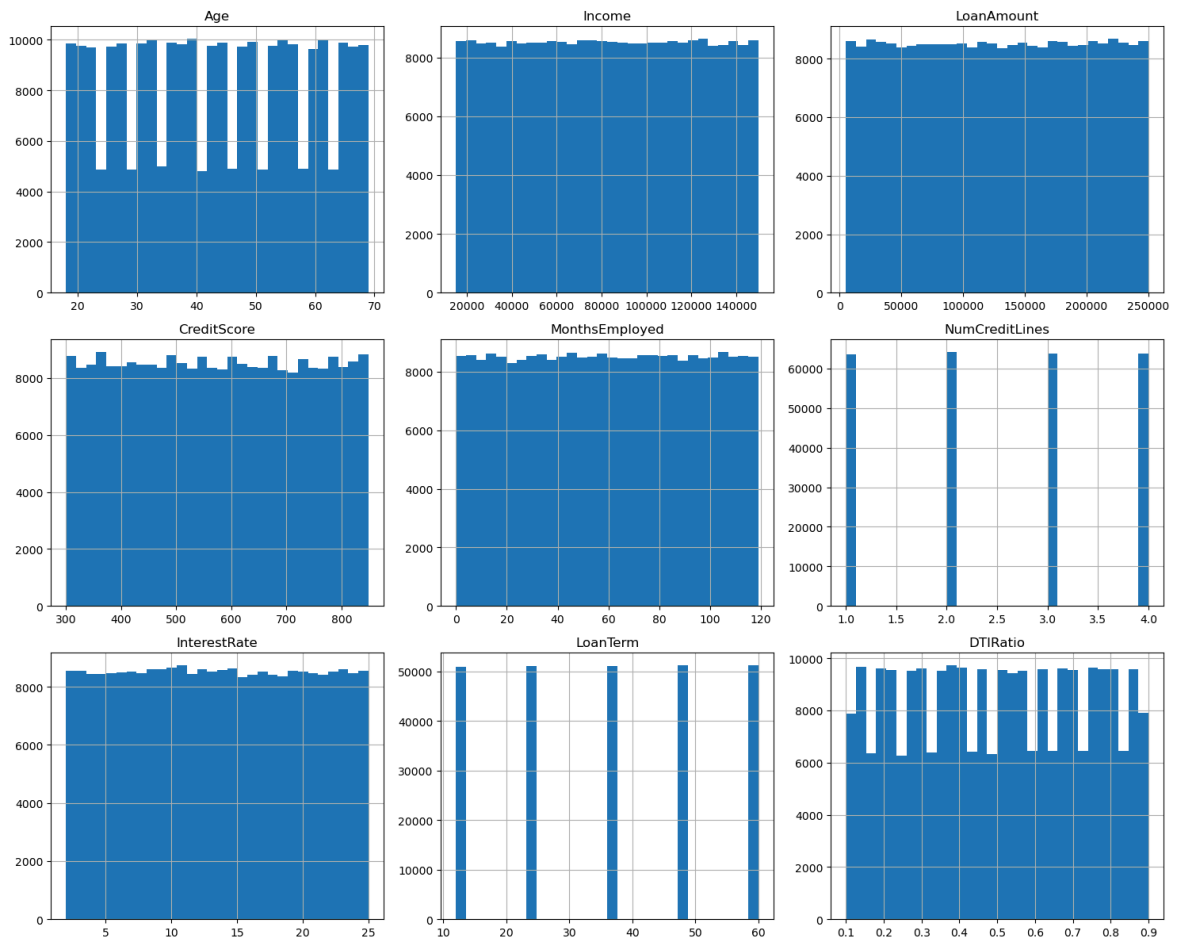
In []:

EDA

Numerical Distribution

```
In [14]: # Numerical Histograms
df_clean[numeric_cols].hist(
    figsize=(15, 12),
    bins=30
)

plt.tight_layout()
plt.show()
```



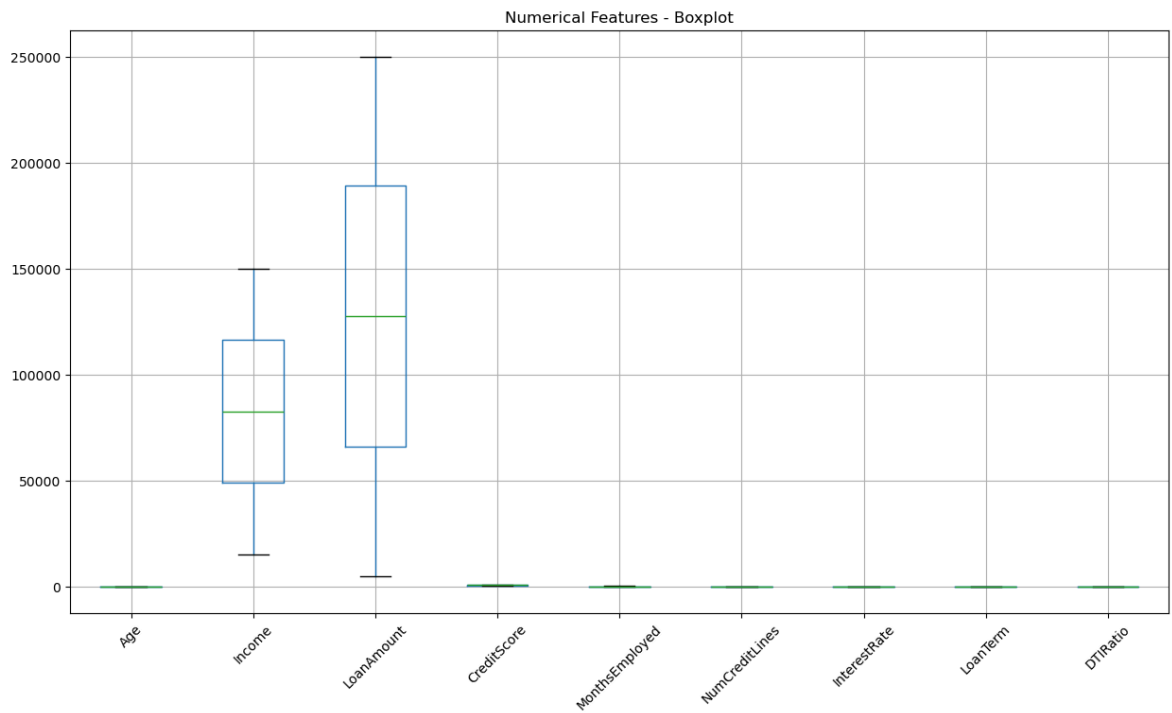
Boxplots

```
In [33]: # Numerical Boxplots
plt.figure(figsize=(15, 8))

df_clean[numeric_cols].boxplot()

plt.xticks(rotation=45)
plt.title('Numerical Features - Boxplot')

plt.show()
```



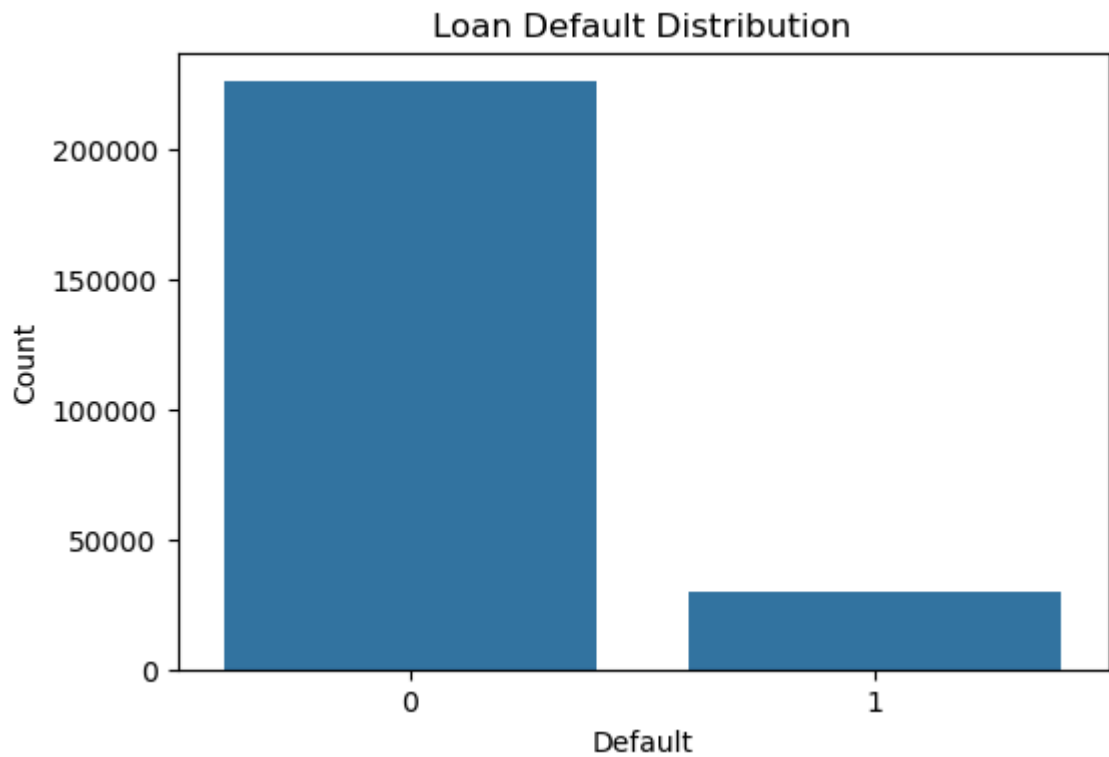
Default Distribution

```
In [34]: # Default Distribution
plt.figure(figsize=(6, 4))

sns.countplot(
    x='Default',
    data=df_clean
)

plt.title('Loan Default Distribution')
plt.xlabel('Default')
plt.ylabel('Count')

plt.show()
```



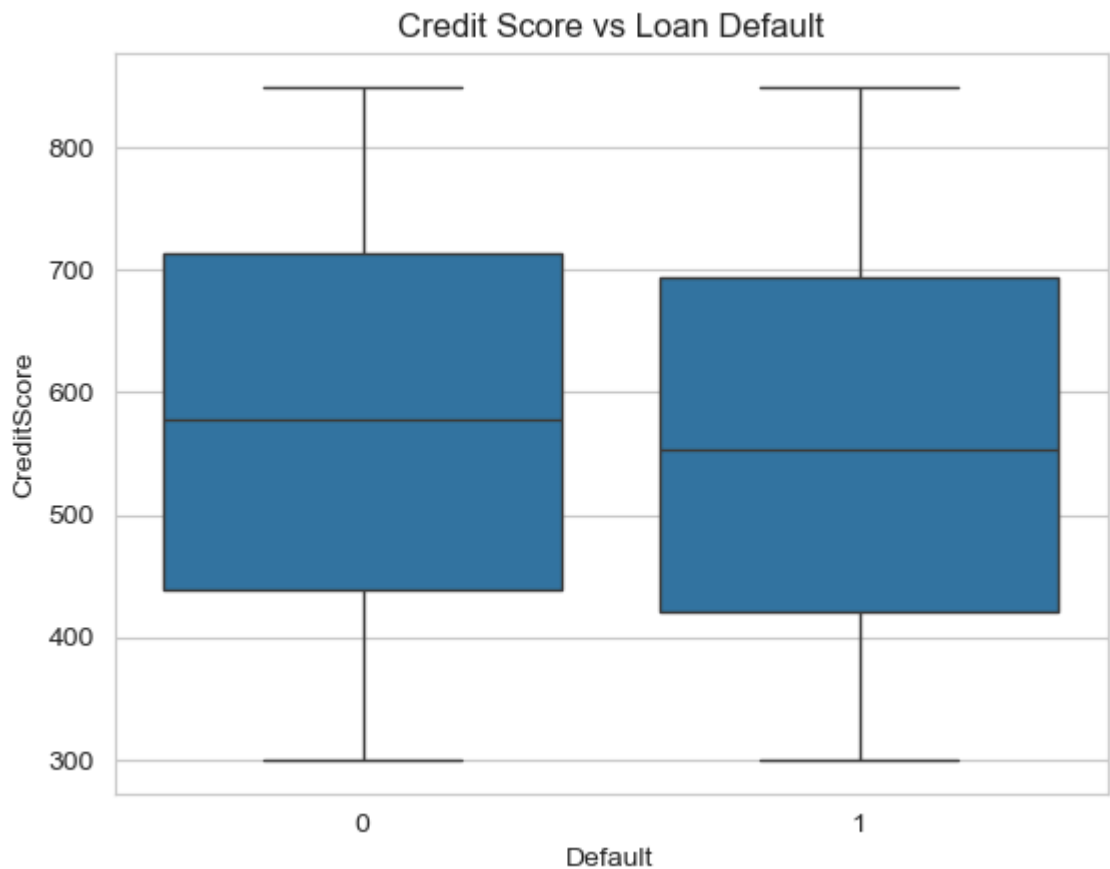
EDA

Credit Score vs Default

```
In [89]: # Credit Score vs Default
sns.boxplot(
    x='Default',
    y='CreditScore',
    data=df_clean
)

plt.title('Credit Score vs Loan Default')

plt.show()
```

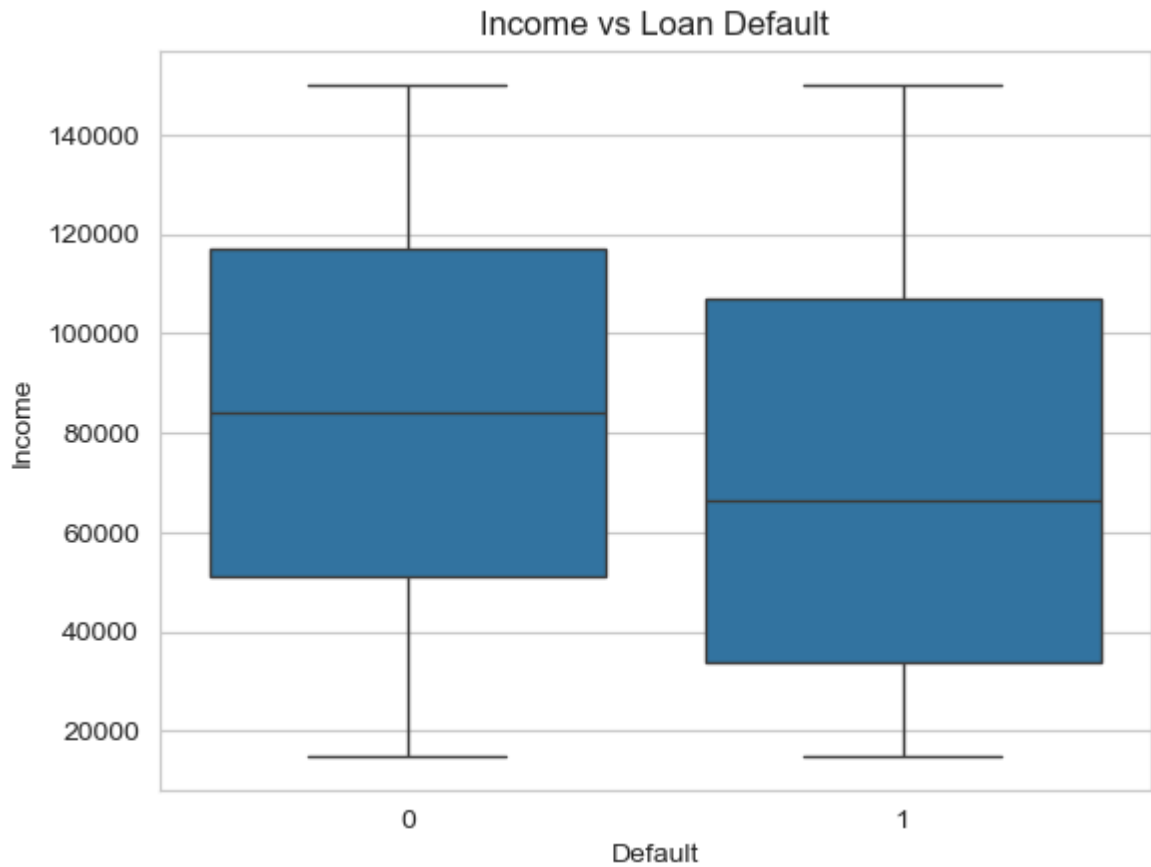


Income vs Default

```
In [91]: # Income vs Default
sns.boxplot(
    x='Default',
    y='Income',
    data=df_clean
)

plt.title('Income vs Loan Default')

plt.show()
```

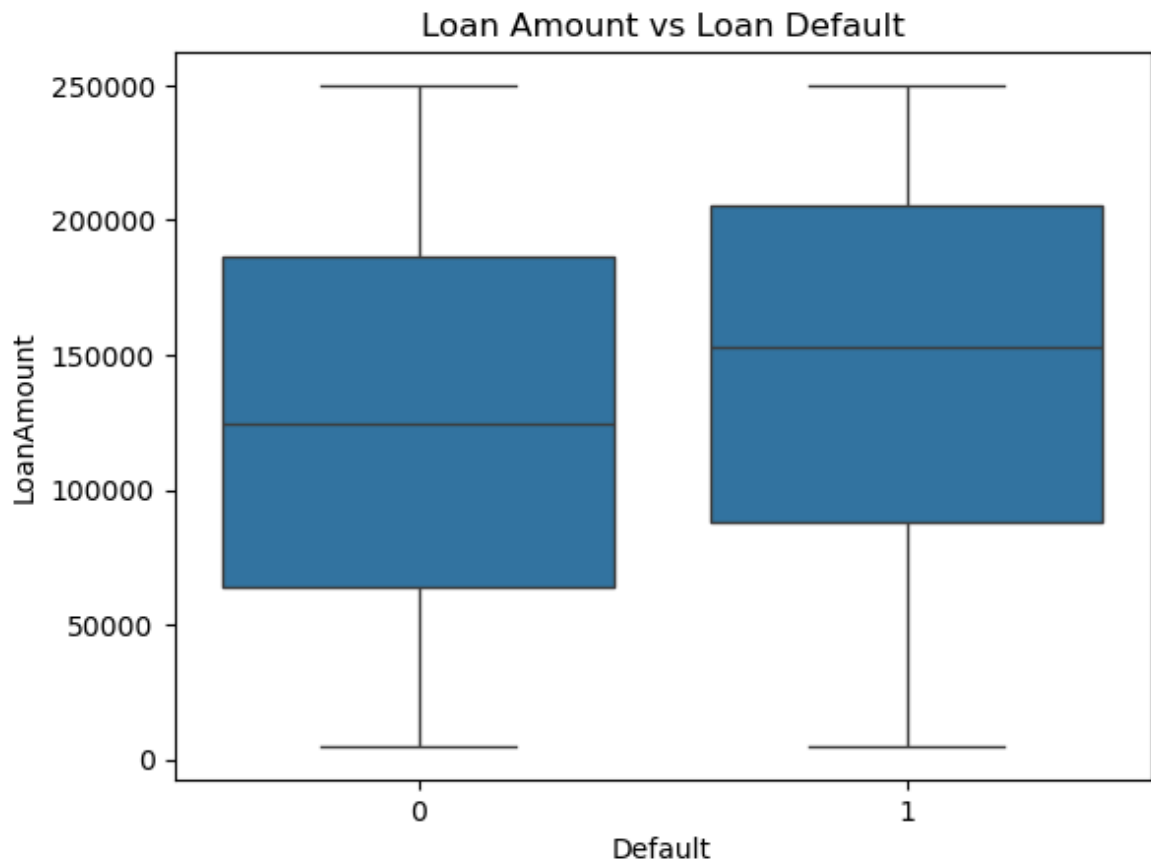


Loan Amount vs Default

```
In [15]: # Loan Amount vs Default
sns.boxplot(
    x='Default',
    y='LoanAmount',
    data=df_clean
)

plt.title('Loan Amount vs Loan Default')

plt.show()
```



In []:

In []:

Week 3 Logistic Regression Using Scikit-Learn.

```
In [17]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)
```

```
In [18]: df_encoded = pd.get_dummies(
    df_clean,
```

```

        columns=categorical_cols,
        drop_first=True
    )

```

In [19]: `df_encoded.head()`

Out[19]:

	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCreditLines	Interest
--	-----	--------	------------	-------------	----------------	----------------	----------

0	56	85994	50587	520	80	4	
1	69	50432	124440	458	15	1	
2	46	84208	129188	451	26	3	
3	32	31713	44799	743	0	3	
4	60	20437	9139	633	8	4	

5 rows × 25 columns



In [20]: `df_encoded.columns`

Out[20]: Index(['Age', 'Income', 'LoanAmount', 'CreditScore', 'MonthsEmployed', 'NumCreditLines', 'InterestRate', 'LoanTerm', 'DTIRatio', 'Default', 'Education_High School', 'Education_Master's', 'Education_PhD', 'EmploymentType_Part-time', 'EmploymentType_Self-employed', 'EmploymentType_Unemployed', 'MaritalStatus_Married', 'MaritalStatus_Single', 'HasMortgage_Yes', 'HasDependents_Yes', 'LoanPurpose_Business', 'LoanPurpose_Education', 'LoanPurpose_Home', 'LoanPurpose_Other', 'HasCoSigner_Yes'], dtype='object')

In [21]: `print("Rows :", df_encoded.shape[0])`
`print("Columns :", df_encoded.shape[1])`

Rows : 255347
Columns : 25

In [22]: `X = df_encoded.drop('Default', axis=1)`
`y = df_encoded['Default']`

In [23]: `print("X Shape :", X.shape)`
`print("y Shape :", y.shape)`

X Shape : (255347, 24)
y Shape : (255347,)

In [24]: `print(y.value_counts())`

Default
0 225694
1 29653
Name: count, dtype: int64

In [25]: `print(y.value_counts(normalize=True) * 100)`

```
Default
0      88.387175
1      11.612825
Name: proportion, dtype: float64
```

Train-Test Split

```
In [26]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )
```

```
In [27]: print("X_train :", X_train.shape)
        print("X_test  :", X_test.shape)
        print("y_train :", y_train.shape)
        print("y_test  :", y_test.shape)
```

```
X_train : (204277, 24)
X_test  : (51070, 24)
y_train : (204277,)
y_test  : (51070,)
```

```
In [28]: scaler = StandardScaler()

        X_train_scaled = scaler.fit_transform(X_train)

        X_test_scaled = scaler.transform(X_test)
```

```
In [29]: # Convert Scaled Data to DataFrame

        X_train_scaled = pd.DataFrame(
            X_train_scaled,
            columns=X_train.columns,
            index=X_train.index
        )

        X_test_scaled = pd.DataFrame(
            X_test_scaled,
            columns=X_test.columns,
            index=X_test.index
        )
```

Create Logistic Regression Model

```
In [30]: model = LogisticRegression(
        max_iter=1000,
        random_state=42
    )
```

```
In [31]: # Train Logistic Regression Model

        model.fit(
```

```
X_train_scaled,  
y_train  
)
```

Out[31]:

```
LogisticRegression  
  
LogisticRegression(max_iter=1000, random_state=42)
```

```
In [32]: print("Intercept :", model.intercept_)  
  
print("Coefficients :", model.coef_)
```

```
Intercept : [-2.37698198]  
Coefficients : [[-0.58487301 -0.34188488  0.3002732  -0.12197679 -0.33702583  0.1  
0038645  
 0.45823645  0.00384927  0.06827958  0.03280022 -0.05781611 -0.07748757  
 0.11868621  0.10556089  0.19249374 -0.09871936 -0.02870227 -0.07754922  
-0.12631903  0.0228807  -0.00354398 -0.07385113 -0.00401992 -0.1311732  ]]
```

```
In [33]: coefficients = pd.DataFrame({  
    'Feature': X_train.columns,  
    'Coefficient': model.coef_[0]  
})  
  
coefficients
```

Out[33]:

	Feature	Coefficient
0	Age	-0.584873
1	Income	-0.341885
2	LoanAmount	0.300273
3	CreditScore	-0.121977
4	MonthsEmployed	-0.337026
5	NumCreditLines	0.100386
6	InterestRate	0.458236
7	LoanTerm	0.003849
8	DTIRatio	0.068280
9	Education_High School	0.032800
10	Education_Master's	-0.057816
11	Education_PhD	-0.077488
12	EmploymentType_Part-time	0.118686
13	EmploymentType_Self-employed	0.105561
14	EmploymentType_Unemployed	0.192494
15	MaritalStatus_Married	-0.098719
16	MaritalStatus_Single	-0.028702
17	HasMortgage_Yes	-0.077549
18	HasDependents_Yes	-0.126319
19	LoanPurpose_Business	0.022881
20	LoanPurpose_Education	-0.003544
21	LoanPurpose_Home	-0.073851
22	LoanPurpose_Other	-0.004020
23	HasCoSigner_Yes	-0.131173

In [34]: `coefficients.sort_values(by='Coefficient', ascending=False)`

Out[34]:

	Feature	Coefficient
6	InterestRate	0.458236
2	LoanAmount	0.300273
14	EmploymentType_Unemployed	0.192494
12	EmploymentType_Part-time	0.118686
13	EmploymentType_Self-employed	0.105561
5	NumCreditLines	0.100386
8	DTIRatio	0.068280
9	Education_High School	0.032800
19	LoanPurpose_Business	0.022881
7	LoanTerm	0.003849
20	LoanPurpose_Education	-0.003544
22	LoanPurpose_Other	-0.004020
16	MaritalStatus_Single	-0.028702
10	Education_Master's	-0.057816
21	LoanPurpose_Home	-0.073851
11	Education_PhD	-0.077488
17	HasMortgage_Yes	-0.077549
15	MaritalStatus_Married	-0.098719
3	CreditScore	-0.121977
18	HasDependents_Yes	-0.126319
23	HasCoSigner_Yes	-0.131173
4	MonthsEmployed	-0.337026
1	Income	-0.341885
0	Age	-0.584873

```
In [35]: y_train_pred = model.predict(X_train_scaled)
```

```
In [36]: y_test_pred = model.predict(X_test_scaled)
```

```
In [37]: y_test_probability = model.predict_proba(X_test_scaled)[: , 1]
```

```
In [38]: prediction_result = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_test_pred,
    'Default_Probability': y_test_probability
})
prediction_result.head(20)
```

Out[38]:

	Actual	Predicted	Default_Probability
0	0	0	0.030263
1	0	0	0.020312
2	0	0	0.075602
3	0	0	0.152385
4	0	0	0.083974
5	0	0	0.009431
6	0	0	0.355224
7	0	0	0.072623
8	1	0	0.337806
9	0	0	0.158059
10	0	0	0.425298
11	0	0	0.025274
12	1	0	0.203439
13	0	0	0.051330
14	0	0	0.013396
15	0	0	0.045082
16	0	0	0.024744
17	0	0	0.064951
18	0	0	0.030917
19	0	0	0.412168

```
In [39]: accuracy = accuracy_score(y_test, y_test_pred)
print("Accuracy :", accuracy)
```

Accuracy : 0.885275112590562

```
In [40]: # Classification Report and Confusion Matrix
```

```
print("Classification Report:")
print(
    classification_report(
        y_test,
        y_test_pred
    )
)

cm = confusion_matrix(
    y_test,
    y_test_pred
)

print("\nConfusion Matrix:")
print(cm)
```

```
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['No Default', 'Default'],
    yticklabels=['No Default', 'Default']
)

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Logistic Regression')

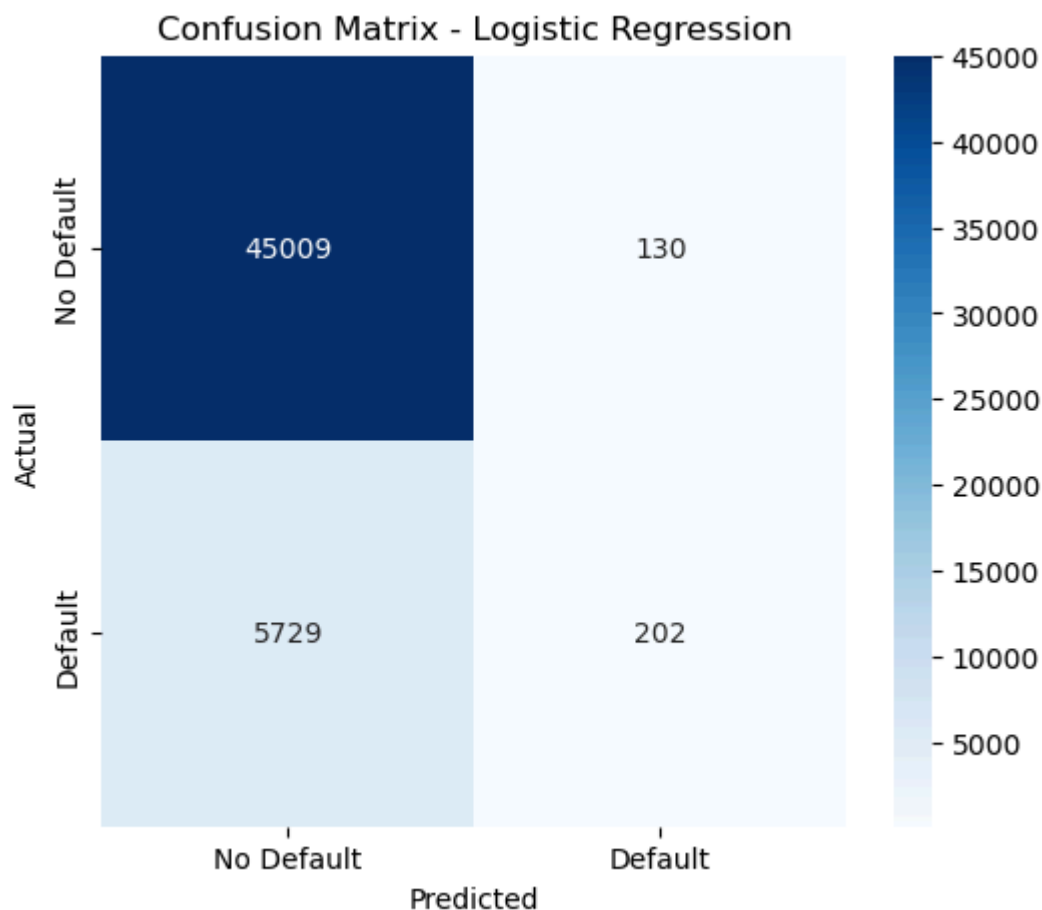
plt.show()
```

Classification Report:

	precision	recall	f1-score	support
0	0.89	1.00	0.94	45139
1	0.61	0.03	0.06	5931
accuracy			0.89	51070
macro avg	0.75	0.52	0.50	51070
weighted avg	0.85	0.89	0.84	51070

Confusion Matrix:

```
[[45009  130]
 [ 5729  202]]
```



In []:

In []:

Week 3 Gradient Descent From Scratch

In [41]: *# Prepare NumPy Arrays*

```
X_train_np = X_train_scaled.values.astype(float)
X_test_np = X_test_scaled.values.astype(float)
y_train_np = y_train.values.astype(float)
y_test_np = y_test.values.astype(float)
```

In [42]:

```
X_train_gd = np.c_[np.ones(X_train_np.shape[0]), X_train_np]
```

```
X_test_gd = np.c_[np.ones(X_test_np.shape[0]), X_test_np]
```

In [43]:

```
n_samples = X_train_gd.shape[0]
```

```
n_features = X_train_gd.shape[1]
```

```
weights = np.zeros(n_features)
```

```
learning_rate = 0.01
```

```
epochs = 1000
```

```
loss_history = []
```

In [44]:

```
def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))
```

In [45]: *# Binary Cross-Entropy Loss*

```
def binary_cross_entropy(y_true, y_pred):

    epsilon = 1e-15

    y_pred = np.clip(
        y_pred,
        epsilon,
        1 - epsilon
    )

    loss = -np.mean(
        y_true * np.log(y_pred)
        +
        (1 - y_true) * np.log(1 - y_pred)
    )

    return loss
```

```
In [46]: # Gradient Descent

for epoch in range(epochs):

    z = np.dot(
        X_train_gd,
        weights
    )

    y_pred = sigmoid(z)

    loss = binary_cross_entropy(
        y_train_np,
        y_pred
    )

    loss_history.append(loss)

    gradient = (
        np.dot(
            X_train_gd.T,
            (y_pred - y_train_np)
        )
        / n_samples
    )

    weights = weights - (
        learning_rate * gradient
    )

    if epoch % 100 == 0:
        print(
            f"Epoch {epoch} | Loss = {loss:.6f}"
        )
```

```
Epoch 0 | Loss = 0.693147
Epoch 100 | Loss = 0.570396
Epoch 200 | Loss = 0.494627
Epoch 300 | Loss = 0.446198
Epoch 400 | Loss = 0.414012
Epoch 500 | Loss = 0.391821
Epoch 600 | Loss = 0.376012
Epoch 700 | Loss = 0.364424
Epoch 800 | Loss = 0.355716
Epoch 900 | Loss = 0.349029
```

```
In [48]: print("Final Loss :", loss_history[-1])
```

```
Final Loss : 0.3438417072527464
```

```
In [50]: print("Final Weights:")
for feature, weight in zip(['Intercept'] + list(X_train.columns), weights):
    print(f"{feature}: {weight}")
```

Final Weights:

Intercept: -1.5891648604491888

Age: -0.26154776895418086

Income: -0.1549485073094603

LoanAmount: 0.13567152901968904

CreditScore: -0.05515655681350721

MonthsEmployed: -0.1519862636633104

NumCreditLines: 0.04467930833477169

InterestRate: 0.20642094690124674

LoanTerm: 0.001558658717374025

DTIRatio: 0.03179970472579007

Education_High School: 0.026004654025781112

Education_Master's: -0.01979505012045812

Education_PhD: -0.028200328417007658

EmploymentType_Part-time: 0.02131091497424106

EmploymentType_Self-employed: 0.013503221805229875

EmploymentType_Unemployed: 0.05988269396494499

MaritalStatus_Married: -0.03814643405331737

MaritalStatus_Single: -0.0025115244601825364

HasMortgage_Yes: -0.03519110791662398

HasDependents_Yes: -0.056591210956904225

LoanPurpose_Business: 0.01598358322650505

LoanPurpose_Education: 0.0031451106109921263

LoanPurpose_Home: -0.03105254735892629

LoanPurpose_Other: 0.0017132584424310133

HasCoSigner_Yes: -0.05946224860549599

In [51]: *# Gradient Descent Prediction*

```
test_probability_gd = sigmoid(
    np.dot(
        X_test_gd,
        weights
    )
)

test_prediction_gd = (
    test_probability_gd >= 0.5
).astype(int)

print(
    "First 20 Predictions:"
)

print(
    test_prediction_gd[:20]
)
```

First 20 Predictions:

[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

In [52]:

```
gd_accuracy = accuracy_score(
    y_test_np,
    test_prediction_gd
)
print("Accuracy : ", gd_accuracy);
```

Accuracy : 0.8838652829449775

In [53]: *# Gradient Descent Loss Curve and Confusion Matrix*

```
plt.figure(figsize=(8, 5))

plt.plot(
    range(epochs),
    loss_history
)

plt.xlabel('Epoch')
plt.ylabel('Binary Cross-Entropy Loss')
plt.title('Gradient Descent Loss Curve')

plt.show()

cm_gd = confusion_matrix(
    y_test_np,
    test_prediction_gd
)

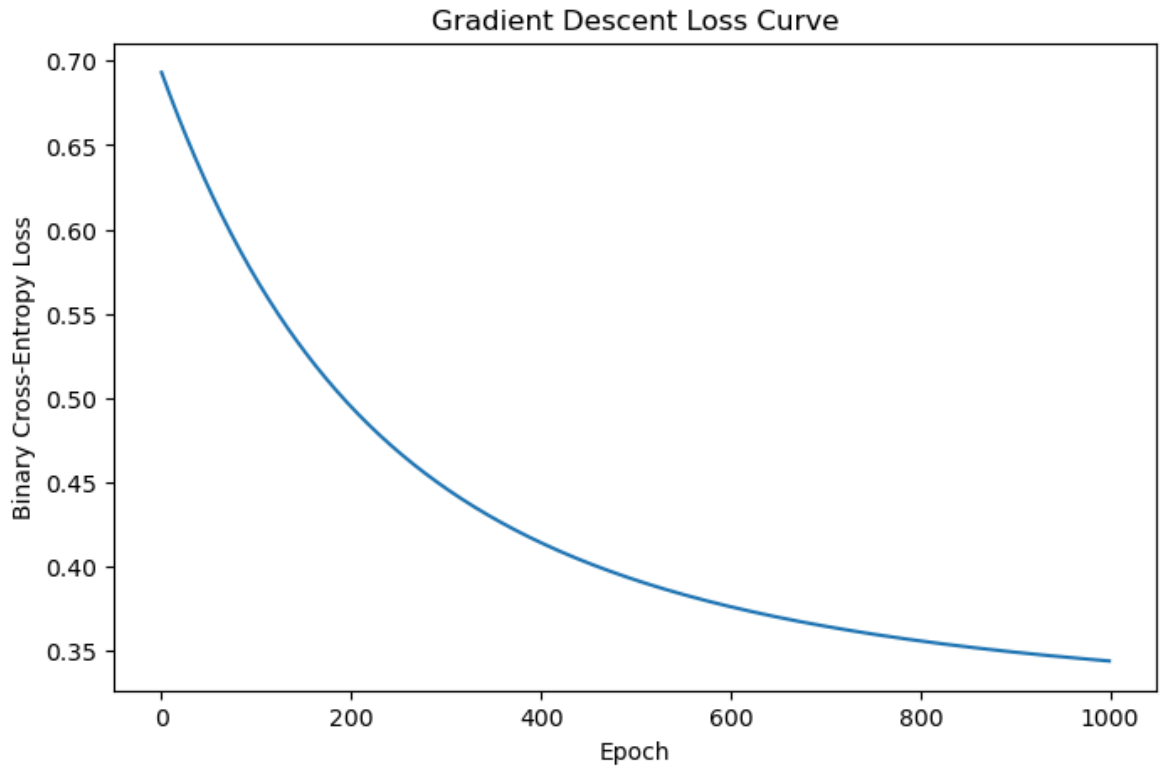
print("Confusion Matrix:")
print(cm_gd)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_gd,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['No Default', 'Default'],
    yticklabels=['No Default', 'Default']
)

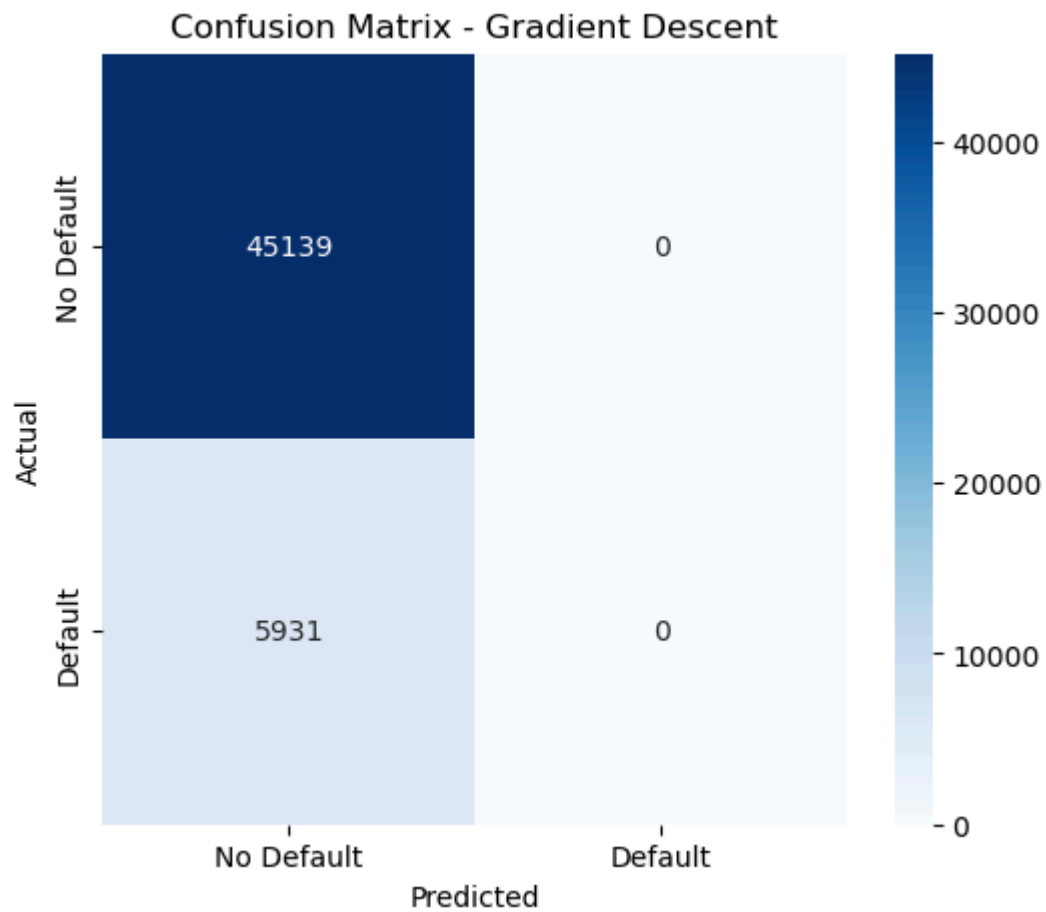
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Gradient Descent')

plt.show()
```



Confusion Matrix:

```
[[45139    0]
 [ 5931    0]]
```



In []:

In []:

In []:

Week 4 — Logistic Regression, KNN, Naive Bayes and Decision Tree Models

```
In [54]: from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
```

1. Logistic Regression Model (with Hyperparameter Tuning)

```
In [84]: param_grid_lr = {
    'C': [0.01, 0.1, 1.0, 10.0],
    'solver': ['liblinear', 'lbfgs']
}

grid_lr = GridSearchCV(
    LogisticRegression(max_iter=1000, random_state=42),
    param_grid_lr,
    cv=3,
    scoring='accuracy',
    n_jobs=1
)

grid_lr.fit(X_train_scaled, y_train)

model_lr = grid_lr.best_estimator_

print(
    "Best Hyperparameters for Logistic Regression:",
    grid_lr.best_params_
)
```

Best Hyperparameters for Logistic Regression: {'C': 0.01, 'solver': 'liblinear'}

```
In [85]: y_pred_lr = model_lr.predict(X_test_scaled)
train_score_lr = model_lr.score(X_train_scaled, y_train)
test_score_lr = accuracy_score(y_test, y_pred_lr)

print("Training Accuracy:", train_score_lr)
print("Testing Accuracy:", test_score_lr)
```

Training Accuracy: 0.8851265683361318
Testing Accuracy: 0.8850793029175641

```
In [86]: print("Classification Report - Logistic Regression:")
print(classification_report(y_test, y_pred_lr))
print("Confusion Matrix - Logistic Regression:")
print(confusion_matrix(y_test, y_pred_lr))
```

Classification Report - Logistic Regression:

	precision	recall	f1-score	support
0	0.89	1.00	0.94	45139
1	0.60	0.03	0.06	5931
accuracy			0.89	51070
macro avg	0.74	0.51	0.50	51070
weighted avg	0.85	0.89	0.84	51070

Confusion Matrix - Logistic Regression:

```
[[45015  124]
 [ 5745  186]]
```

In []:

In []:

In [50]:

```
import os

os.environ["LOKY_MAX_CPU_COUNT"] = "1"
```

2. K-Nearest Neighbors (KNN) Model (with Hyperparameter Tuning)

In [63]:

```
from sklearn.neighbors import KNeighborsClassifier

param_grid_knn = {
    'n_neighbors': [5, 9, 15],
    'weights': ['uniform', 'distance']
}

X_train_sub = X_train_scaled.sample(30000, random_state=42)
y_train_sub = y_train.loc[X_train_sub.index]

grid_knn = GridSearchCV(
    KNeighborsClassifier(),
    param_grid_knn,
    cv=3,
    scoring='accuracy',
    n_jobs=-1
)
grid_knn.fit(X_train_sub, y_train_sub)

print("Best Hyperparameters for KNN:", grid_knn.best_params_)

model_knn = grid_knn.best_estimator_
model_knn.fit(X_train_scaled, y_train)
```

Best Hyperparameters for KNN: {'n_neighbors': 15, 'weights': 'uniform'}

Out[63]:

```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=15)
```

```
In [64]: y_pred_knn = model_knn.predict(X_test_scaled)
train_score_knn = model_knn.score(X_train_scaled, y_train)
test_score_knn = accuracy_score(y_test, y_pred_knn)

print("Training Accuracy:", train_score_knn)
print("Testing Accuracy:", test_score_knn)
```

Training Accuracy: 0.8851804167870098

Testing Accuracy: 0.8836498923046798

```
In [68]: print("Classification Report - KNN:")
print(classification_report(y_test, y_pred_knn))
print("Confusion Matrix - KNN:")
print(confusion_matrix(y_test, y_pred_knn))
```

Classification Report - KNN:

	precision	recall	f1-score	support
0	0.88	1.00	0.94	45139
1	0.45	0.01	0.02	5931
accuracy			0.88	51070
macro avg	0.67	0.50	0.48	51070
weighted avg	0.83	0.88	0.83	51070

Confusion Matrix - KNN:

```
[[45077  62]
 [ 5880  51]]
```

In []:

In []:

3. Naive Bayes Model (with Hyperparameter Tuning)

```
In [75]: param_grid_nb = {
        'var_smoothing': np.logspace(0, -9, num=10).tolist()
    }

    grid_nb = GridSearchCV(
        GaussianNB(),
        param_grid_nb,
        cv=3,
        scoring='accuracy',
        n_jobs=-1
    )
    grid_nb.fit(X_train_scaled, y_train)

    model_nb = grid_nb.best_estimator_
    print("Best Hyperparameters for Naive Bayes:", grid_nb.best_params_)
```

Best Hyperparameters for Naive Bayes: {'var_smoothing': 0.01}

```
In [77]: y_pred_nb = model_nb.predict(X_test_scaled)
train_score_nb = model_nb.score(X_train_scaled, y_train)
test_score_nb = accuracy_score(y_test, y_pred_nb)
```

```
print("Training Accuracy:", train_score_nb)
print("Testing Accuracy:", test_score_nb)
```

Training Accuracy: 0.884803477630864

Testing Accuracy: 0.8849422361464656

```
In [78]: print("Classification Report - Naive Bayes:")
print(classification_report(y_test, y_pred_nb))
print("Confusion Matrix - Naive Bayes:")
print(confusion_matrix(y_test, y_pred_nb))
```

Classification Report - Naive Bayes:

	precision	recall	f1-score	support
0	0.89	1.00	0.94	45139
1	0.62	0.02	0.05	5931
accuracy			0.88	51070
macro avg	0.75	0.51	0.49	51070
weighted avg	0.86	0.88	0.84	51070

Confusion Matrix - Naive Bayes:

```
[[45054  85]
 [ 5791 140]]
```

In []:

In []:

4. Decision Tree Model (with Hyperparameter Tuning)

```
In [79]: param_grid_dt = {
    'max_depth': [3, 5, 8, 10],
    'min_samples_split': [10, 20, 50],
    'min_samples_leaf': [5, 10, 20]
}

grid_dt = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid_dt,
    cv=3,
    scoring='accuracy',
    n_jobs=-1
)
grid_dt.fit(X_train_scaled, y_train)

model_dt = grid_dt.best_estimator_
print("Best Hyperparameters for Decision Tree:", grid_dt.best_params_)
```

Best Hyperparameters for Decision Tree: {'max_depth': 5, 'min_samples_leaf': 5, 'min_samples_split': 10}

```
In [80]: y_pred_dt = model_dt.predict(X_test_scaled)
train_score_dt = model_dt.score(X_train_scaled, y_train)
test_score_dt = accuracy_score(y_test, y_pred_dt)
```

```
print("Training Accuracy:", train_score_dt)
print("Testing Accuracy:", test_score_dt)
```

Training Accuracy: 0.8848524307680258

Testing Accuracy: 0.8848834932445663

```
In [81]: print("Classification Report - Decision Tree:")
print(classification_report(y_test, y_pred_dt))
print("Confusion Matrix - Decision Tree:")
print(confusion_matrix(y_test, y_pred_dt))
```

Classification Report - Decision Tree:

	precision	recall	f1-score	support
0	0.89	1.00	0.94	45139
1	0.63	0.02	0.04	5931
accuracy			0.88	51070
macro avg	0.76	0.51	0.49	51070
weighted avg	0.86	0.88	0.83	51070

Confusion Matrix - Decision Tree:

```
[[45062   77]
 [ 5802  129]]
```

In []:

In []:

Model Comparison

```
In [87]: results_df = pd.DataFrame({
    'Model': ['Logistic Regression', 'K-Nearest Neighbors (KNN)', 'Naive Bayes',
    'Training Accuracy': [train_score_lr, train_score_knn, train_score_nb, train
    'Testing Accuracy': [test_score_lr, test_score_knn, test_score_nb, test_score_dt]
})

results_df
```

Out[87]:

	Model	Training Accuracy	Testing Accuracy
0	Logistic Regression	0.885127	0.885079
1	K-Nearest Neighbors (KNN)	0.885180	0.883650
2	Naive Bayes	0.884803	0.884942
3	Decision Tree	0.884852	0.884883

In []: